Guidelines to Create a Simulation

1. Goals and Ideas

Before writing code, decide exactly what you want the student to learn

Source : Study school curriculum state syllabus

- **Focus on Hard Concepts:** Find topics that students usually confuse (like heat versus temperature) and build the game around clearing up that confusion.
- **Show Hidden Things:** Show things that are impossible to see in real life, like moving molecules, tiny atoms, or flowing electricity.
- **Link Different Views:** When a student changes something, show the results in multiple ways at the same time. For example, show an animation moving faster AND a chart going up at the exact same moment.

> **RULE FOR GOALS:**
> You MUST show the invisible parts of science (like atoms, energy, or forces) so students can see how they work.

2. Screen and Buttons (UI)

A good simulation does not need an instruction manual. The design should guide the user naturally.

- **No Instructions:** Do not put long text rules or steps on the screen. Let the buttons and tools show the student what to do.
- **Use Familiar Objects:** Use everyday items that players already understand. For example, use a bicycle pump to add air or a water tap to fill a container.
- **Start Simple:** When the simulation opens, keep the screen clean, quiet, or paused. Too many moving objects at the start can overwhelm students.
- **Use Very Few Words:** Only use words for small labels on tools or buttons. Let the action do the teaching.

> **RULE FOR INTERFACES:**
> The simulation MUST start in a very simple, clean, or paused state with no long instructions. The design must be clear enough for anyone to use immediately.

3. How the Simulation Works

Expected Outcome : Simulation should run in single screen and view like a game

- **Instant Reaction:** Whenever a user moves a tool or changes a slider, the simulation must react instantly. Fast feedback helps students learn cause and effect.
- **Give Superpowers:** Let students do things they cannot do in a real classroom. Allow them to pause time, slow down fast actions, or change things like gravity.
- **Make a Measurement Game:** Do not show all the data and answers immediately. Make students drag tools (like a ruler or a thermometer) into the screen to find out the answers themselves.

> **RULE FOR MECHANICS:**
> The tools MUST react immediately without lagging. Give students special tools to freeze time or change science constants to help them test ideas.

4. Standard Technology Rules (Tech Stack)

To make sure all published simulations work exactly the same way, stay light, and look sharp on all screens, developers must follow these strict rules:

- **Core Languages (HTML5, CSS, JS):** Build the simulation entirely with standard web code: HTML5 for structure, CSS3 for layouts, and clean JavaScript (JS) for the math and behavior. Avoid heavy outside software frameworks so that the files load instantly.
- **Vector Graphics Only (SVG):** All buttons, icons, scales, and artwork must use the SVG format. Unlike regular images, SVGs can scale infinitely up or down without becoming blurry, keeping the layout perfectly crisp on high-resolution screens and mobile phones.
- **Canvas and WebGL for Speed:** For heavy, high-speed graphics—such as thousands of moving atoms or complex fluid vectors—use the HTML5 Canvas element or WebGL. This keeps the performance locked at a smooth, lag-free speed on lower-end devices.

> **RULE FOR CODE UNIFORMITY:**
> Every single simulation MUST be built using HTML5, CSS3, and JavaScript, with artwork saved strictly as SVGs. This is required so all projects behave exactly the same way once published.

5. Making it for Everyone

- **Easy to See and Control:** Use colors that are easy for color-blind students to see. Make sure the simulation can be controlled using just a computer keyboard instead of a mouse.
- **Add Sounds:** Use helpful sound effects. For example, make a sound go higher in pitch when the pressure increases inside a tank.
- **Works on Any Device:** Build the simulation so it works perfectly on expensive computers, school Chromebooks, or basic mobile phones.

6. Simple End-of-Simulation Assessment

Include a fun, low-stakes checking screen at the very end to help students verify what they learned. This section should feel like a game, not a stressful test.

- **The 'Game Screen' Concept:** Instead of a formal test, build a distinct 'Game' or 'Challenge' tab at the end. Use interactive puzzle challenges (e.g., 'Can you adjust the variables to make the balance level?') rather than basic text multiple-choice questions.
- **Immediate Correction and Retries:** When a student answers a question or puzzle incorrectly, do not simply mark a red 'X'. Tell them why it didn't work and allow them to jump back into the simulation to test it again instantly.
- **Reward Exploration, Not Just Scores:** Focus on completion milestones and stars earned rather than a rigid percentage grade. The goal is to make the student feel encouraged to retry a concept until they get it right.

> **RULE FOR ASSESSMENT:**
> The end assessment MUST be designed as an interactive game rather than a text-based exam. It must provide instant visual explanations for mistakes and give students infinite chances to retry.

7. Testing and Improving

- **Watch Students Play:** Before publishing, test the simulation with real students. Sit back and watch them use it without giving them any tips. If they get confused, rewrite the code or fix the layout.